

Introduction to Algorithms II

Edit Distance and Greedy Algorithms

Hoa T. Vu

Edit Distance

Edit Distance : Problem

Given two strings s and t , find the **minimum number of operations** to transform s into t . Allowed operations:

- **Insert** a character
- **Delete** a character
- **Substitute** a character

Example: $s = \text{"horse"}$, $t = \text{"ros"}$

Edit Distance : Problem

Given two strings s and t , find the **minimum number of operations** to transform s into t . Allowed operations:

- **Insert** a character
- **Delete** a character
- **Substitute** a character

Example: $s = \text{"horse"}$, $t = \text{"ros"}$

horse $\xrightarrow{\text{delete h}}$ orse $\xrightarrow{\text{substitute o} \rightarrow \text{r}}$ rrse $\xrightarrow{\text{delete e}}$ rrs

Edit Distance : Placing the Gaps

Line up *s* and *t* side by side, placing a **gap** (–) wherever a character was inserted or deleted in the transformation above:

<i>s</i> :	h	o	r	s	e
<i>t</i> :	–	r	r	s	–

- **match** (same letter): cost 0
- **substitute** (letters differ): cost 1
- **gap** (insert or delete): cost 1

Total cost = $1 + 1 + 0 + 0 + 1 = 3$ edits.

Edit Distance : Subproblem

Subproblem: $dp[i][j]$ = edit distance between $s[:i]$ (first i characters of s) and $t[:j]$ (first j characters of t).

Goal: compute $dp[m][n]$ where $m = |s|$, $n = |t|$.

Edit Distance : Subproblem

Subproblem: $dp[i][j]$ = edit distance between $s[:i]$ (first i characters of s) and $t[:j]$ (first j characters of t).

Goal: compute $dp[m][n]$ where $m = |s|$, $n = |t|$. **Base cases:**

$$dp[i][0] = i \quad dp[0][j] = j$$

Transforming a string of length i into the empty string takes i deletions (and symmetrically for insertions).

Edit Distance : Comparing Characters

To compute $dp[i][j]$, compare the last characters of the two prefixes: $s[i - 1]$ and $t[j - 1]$.

- If they **match**, no operation is needed here.
- If they **differ**, we must pay for an operation : insert, delete, or substitute.

Each case gives a different way to compute $dp[i][j]$ from smaller subproblems.

Edit Distance : Case 1 – Match

Case 1: $s[i - 1] = t[j - 1]$. The last column of the alignment has the **same character** on both rows : no gap needed:

s:	...	x
t:	...	x

Meaning: the last characters already agree, so this column is free. Everything left to resolve is the alignment of the prefixes $s[: i - 1]$ and $t[: j - 1]$:

$$dp[i][j] = dp[i-1][j-1]$$

Edit Distance : Case 2 – Mismatch

Case 2: $s[i - 1] \neq t[j - 1]$. The last column cannot be a free match, so it must cost

1. There are three ways to fill that column:

- **Delete:** gap below, $s[i - 1]$ above
- **Insert:** gap above, $t[j - 1]$ below
- **Substitute:** different characters, no gap

Try all three and take the cheapest.

Edit Distance : Mismatch – Delete

Last column: $s[i - 1]$ above, a **gap** below : t contributes nothing here.

s :	...	x
t :	...	-

Meaning: $s[i - 1]$ is deleted, with no partner in t . What's left to resolve is the prefix $s[: i - 1]$ against *all* of $t[: j]$:

$$dp[i][j] = 1 + dp[i-1][j]$$

Edit Distance : Mismatch – Insert

Last column: a **gap** above : s contributes nothing here, $t[j - 1]$ below.

s :	...	-
t :	...	x

Meaning: $t[j - 1]$ is inserted, with no partner in s . What's left to resolve is *all* of $s[: i]$ against the prefix $t[: j - 1]$:

$$dp[i][j] = 1 + dp[i][j-1]$$

Edit Distance : Mismatch – Substitute

Last column: **different characters** on both rows, no gap : the positions are paired up, just relabeled.

s :	$\dots$	x
t :	$\dots$	y

Meaning: $s[i - 1]$ is relabeled to $t[j - 1]$. The two strings now agree here, so what's left to resolve is the prefix $s[: i - 1]$ against the prefix $t[: j - 1]$:

$$dp[i][j] = 1 + dp[i-1][j-1]$$

Edit Distance : Recurrence (2/2)

Combined recurrence:

$$\text{dp}[i][j] = \begin{cases} \text{dp}[i-1][j-1] & \text{if } s[i-1] = t[j-1] \\ 1 + \min(\text{dp}[i-1][j], \text{dp}[i][j-1], \text{dp}[i-1][j-1]) & \text{otherwise} \end{cases}$$

Fill the table `dp` row by row, using only previously computed entries.

Edit Distance : Example Table

$s = \text{"horse"}, t = \text{"ros"}$

	" "	r	o	s
" "	0	1	2	3
h	1	1	2	3
o	2	2	1	2
r	3	2	2	2
s	4	3	3	2
e	5	4	4	3

Bottom-right entry: $dp[5][3] = 3$: matches our earlier example.

Greedy Algorithms: Interval Selection

Greedy Algorithms

- At each step, make the choice that looks **best right now**
- Fast and simple but only gives the **optimal** answer for problems with the right structure.

Interval Selection : Problem

You are given n intervals (e.g. meeting requests), each with a **start** and **finish** time. Two intervals **conflict** if they overlap.

Goal: select the **maximum number** of non-overlapping intervals.

Example:

Interval	A	B	C	D	E
Start	1	2	4	1	5
Finish	3	5	7	8	9

Interval Selection : Tempting (Wrong) Strategies

- **Shortest interval first?** A short interval can still block two longer, non-overlapping intervals on either side of it.
- **Earliest start time first?** An interval that starts early might also finish very late, blocking many others.

Neither rule is always optimal. We need a smarter greedy choice.

Interval Selection : Greedy Rule

Greedy strategy: sort intervals by **finish time**, earliest first. Scan through and **pick an interval** whenever it doesn't conflict with the last one picked.

Why finish time? Picking the interval that finishes soonest leaves the **most room** on the timeline for future choices : no other valid first choice can do better.

Interval Selection : Example

Sorted by finish time: A(1,3), B(2,5), C(4,7), D(1,8), E(5,9).

Interval	A	B	C	D	E
Start	1	2	4	1	5
Finish	3	5	7	8	9
Decision	pick	skip	pick	skip	skip

Result: {A, C} : 2 non-overlapping intervals.

Interval Selection : Exchange Argument (1/2)

Claim: some optimal solution includes g_1 , the interval with the smallest finish time overall.

Let $O = \{o_1, o_2, \dots, o_m\}$ be *any* optimal solution, listed in order of finish time. If $o_1 = g_1$, we're done.

Otherwise, swap them. Since g_1 has the smallest finish time of *any* interval, $\text{finish}(g_1) \leq \text{finish}(o_1)$. Because o_1 doesn't overlap o_2 in O :

$$\text{start}(o_2) \geq \text{finish}(o_1) \geq \text{finish}(g_1)$$

so g_1 doesn't conflict with $o_2, \dots, o_m$ either.

Interval Selection : Exchange Argument (2/2)

Replacing o_1 with g_1 gives

$$O' = \{g_1, o_2, \dots, o_m\}$$

still a valid set of m non-overlapping intervals : so O' is also optimal, and now contains g_1 .

Recurse: after picking g_1 , the remaining problem is exactly "select the max non-overlapping intervals among those that start after g_1 finishes" : the same problem on a smaller input. By induction, greedy's choice at every later step can also be swapped into an optimal solution the same way.

Conclusion: greedy's entire sequence of picks is optimal.